

Cómputo Concurrente 2026-2

Solución — Práctica 5

Implementaciones de Contadores: Balance entre Corrección y Eficiencia

Profa: Gilde Valeria Rodríguez

Ejercicio 1: El Sloppy Counter Modificado

La modificación propuesta añade un volcado del localCount del hilo que invoca fetch() antes de retornar el valor global:

```
public long fetch() {
    long currentLocal = localCount.get();
    if (currentLocal > 0) {
        globalCount.addAndGet(currentLocal);    // vuelca SOLO el hilo que llama
        localCount.set(0L);
    }
    return globalCount.get();
}
```

Listing 1: Modificación a fetch() (reproducida de la práctica).

1.1. Respuesta: la estructura permanece Eventualmente Consistente

Conclusión: La modificación *no* eleva el nivel de consistencia. La estructura continúa siendo **Eventualmente Consistente** y no alcanza ni la Consistencia Secuencial ni la Linealizabilidad.

1.2. Argumento: ¿por qué NO es Secuencialmente Consistente?

Para que un sistema sea secuencialmente consistente debe existir un orden total de todas las operaciones que respete el *orden de programa* de cada hilo individual. La modificación falla ante el siguiente escenario concreto:

- **Hilo A** realiza 50 incrementos. Su localCount queda en 50 (por debajo del umbral de 100, nunca volcó al global).
- **Hilo A** escribe bandera = true.
- **Hilo B** observa bandera == true y llama a fetch().
- **Hilo B** ejecuta la versión modificada: vuelca su *propio* localCount (que es 0 porque B no ha incrementado) y retorna globalCount.get().
- El resultado es **0**, aunque el Hilo A realizó 50 incrementos antes de levantar la bandera.

En cualquier historia secuencial válida, si B observó el efecto de la escritura bandera = true de A, está obligado a observar también todas las operaciones que A realizó *antes* de esa escritura. Como B lee 0 en lugar de ≥ 50 , no existe ningún orden secuencial global que explique esta ejecución. La consistencia secuencial queda rota.

1.3. Argumento: ¿por qué NO es Linealizable?

La linealizabilidad exige que cada operación parezca ejecutarse de forma instantánea en un único punto en el tiempo real. Para que `fetch()` fuera linealizable, su punto de linealización debería capturar el total *real* de todos los incrementos realizados hasta ese instante. Pero la modificación solo vacía el `localCount` del hilo que invoca `fetch()`; los contadores locales de todos los demás hilos activos permanecen intactos y no son visibles en el valor retornado. No hay un instante único en el que el resultado sea correcto.

1.4. ¿Por qué sigue siendo Eventualmente Consistente?

La Consistencia Eventual garantiza que, si cesan todas las actualizaciones y cada hilo vuelca su estado local, todas las réplicas convergerán al mismo valor correcto. La modificación no altera este comportamiento: si todos los hilos dejaran de incrementar y cada uno llamara a `fetch()` (o se forzara un volcado global), el contador global reflejaría el total exacto. Lo que la modificación sí mejora marginalmente es que el hilo que consulta aporta su propio conteo pendiente, reduciendo la discrepancia observable por ese hilo en particular — pero no la de los demás.

Ejercicio 2: Lectura Libre de Obstrucciones en el Árbol

2a. Implementación con Double-Collect

La técnica de **Double-Collect** (Sección 4.3 de Herlihy & Shavit [HS08]) consiste en recolectar el estado dos veces consecutivas. Si ambas colecciones son idénticas, ningún escritor modificó una hoja entre las dos lecturas, y la instantánea es consistente. Si difieren, al menos un escritor estuvo activo durante el escaneo y se reintenta.

```
import java.util.Arrays;

public class ShavitTreeCounter {

    // ... (estructura de Node y buildTree sin cambios) ...

    // ■■ Recolecta los valores de todas las hojas en un arreglo ■■■■■■■■■■
    private long[] collectLeaves() {
        int numLeaves = (int) Math.pow(2, depth);    // 2^depth hojas
        long[] snapshot = new long[numLeaves];
        collectHelper(root, snapshot, 0);
        return snapshot;
    }

    private int collectHelper(Node node, long[] out, int idx) {
        if (node.isLeaf) {
            out[idx] = node.count.get();              // lectura atómica
            return idx + 1;
        }
        idx = collectHelper(node.left, out, idx);
        idx = collectHelper(node.right, out, idx);
        return idx;
    }

    // ■■ fetch() con Double-Collect: obstruction-free y linealizable ■■■■■
```

```

    public long fetch() {
        while (true) {
            long[] first = collectLeaves();           // primera recolección
            long[] second = collectLeaves();          // segunda recolección

            if (Arrays.equals(first, second)) {
                // Ninguna hoja cambió entre ambas lecturas:
                // el snapshot es consistente → punto de linealización aquí
                long sum = 0;
                for (long v : second) sum += v;
                return sum;
            }
            // Algún escritor interfirió → reintentar
        }
    }
}

```

Listing 2: ShavitTreeCounter con fetch() Double-Collect.

Nota: Para que el campo `depth` esté disponible, se almacena como atributo de instancia en el constructor:

```

    private final int depth;

    public ShavitTreeCounter(int depth) {
        this.depth = depth;
        this.root = buildTree(depth);
    }
}

```

Listing 3: Ajuste al constructor.

2b. Descripción paso a paso del algoritmo

1. **Primera recolección** — `collectLeaves()` recorre el árbol en profundidad y llama a `node.count.get()` en cada hoja. Cada `get()` es atómica (`AtomicLong`), pero las lecturas de hojas distintas no son atómicas entre sí: un escritor puede modificar una hoja entre dos lecturas consecutivas.
2. **Segunda recolección** — Se repite el mismo recorrido inmediatamente. Si ningún hilo escritor ejecutó un `incrementAndGet()` en ninguna hoja durante el intervalo entre ambas colecciones, los dos arreglos serán idénticos.
3. **Comparación** — `Arrays.equals(first, second)` verifica hoja por hoja. Si todos los valores coinciden, el escaneo fue libre de interferencias: el estado que vemos es consistente y se retorna la suma.
4. **Reintento** — Si al menos un valor difiere, algún escritor modificó una hoja mientras leíamos. Tiramos ambas colecciones y comenzamos de nuevo desde el paso 1. Los hilos escritores no se detienen ni son notificados: siguen incrementando normalmente.

El algoritmo no requiere candados (`synchronized`, `Lock`) ni variables volátiles adicionales más allá de las que ya provee `AtomicLong`. Los hilos escritores y el lector no se bloquean mutuamente en ningún momento.

2c. Argumentos teóricos

¿Por qué es Obstruction-Free?

Un método es *obstruction-free* si un hilo que ejecuta en aislamiento (todos los demás hilos están suspendidos) termina en un número finito de pasos [HS08, Def. 3.3.1].

En aislamiento, ningún escritor puede modificar ninguna hoja. Por tanto, la primera y la segunda colección retornarán exactamente los mismos valores: `Arrays.equals` devolverá `true` en la primera iteración del bucle y el método terminará. El número de pasos es $O(n)$ donde n es el número de hojas (2^{depth}), lo cual es finito.

¿Por qué NO es Wait-Free?

Un método es *wait-free* si *todo* hilo termina en un número finito de pasos independientemente del comportamiento de los demás [HS08, Def. 3.3.1]. Esta propiedad más fuerte no se cumple aquí:

Contraejemplo: Supón un árbol con dos hojas y un escritor que incrementa la hoja derecha de forma continua. El lector termina su primera colección y justo antes de iniciar la segunda, el escritor actualiza la hoja derecha. El lector completa la segunda colección y detecta una diferencia → reintenta. Si el escritor repite este patrón de forma adversarial en cada intento del lector, el lector puede reintentar indefinidamente. No existe una cota superior en el número de pasos que el lector necesita realizar.

En la práctica, este escenario adversarial es improbable con planificadores justos, pero teóricamente no puede descartarse. Convertirlo en *wait-free* requeriría un mecanismo de ayuda (*helping*): los escritores anunciarían sus operaciones pendientes y el lector las incorporaría directamente, pero eso aumenta significativamente la complejidad del algoritmo.

¿Por qué el valor retornado es Linealizable? — Punto de Linealización

Recordemos la definición: un método es *linealizable* si cada ejecución tiene un punto de linealización (un instante dentro del intervalo [invocación, respuesta]) tal que el valor retornado es consistente con el estado del objeto en ese instante [HS08, Def. 3.4.1].

Punto de linealización de `fetch()`: el instante en que se completa la última lectura de la segunda colección exitosa (es decir, el último `node.count.get()` de la segunda pasada antes de que `Arrays.equals` retorne `true`).

Justificación formal: si `Arrays.equals(first, second)` retorna `true`, significa que ninguna hoja cambió entre el inicio de la primera colección y el fin de la segunda. En particular, el estado de cada hoja al final de la segunda colección es idéntico al que tenía al inicio de la primera. Por lo tanto, existe un instante real — situado dentro del intervalo de la segunda colección — en el que el árbol exhibía exactamente ese estado de forma simultánea. La suma que retornamos es el valor que el contador tenía en ese instante, lo que satisface la condición de linealizabilidad.

Dicho de otra forma: cuando ambas colecciones coinciden, el lector ha observado un *quiesce instantáneo implícito* — no porque los escritores se hayan detenido, sino porque ninguno modificó ninguna hoja durante el intervalo de lectura. El estado capturado es real y consistente, no una

interpolación.

Referencias

[HS08] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008. — Sección 4.3 (Double-Collect / Atomic Snapshots), Sección 3.3 (Obstruction-Free, Wait-Free), Sección 3.4 (Linearizability).

[HW90] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, 1990.

[Sha11] Nir Shavit. Data structures in the multicore age. *Commun. ACM*, 54(3):76–84, 2011.

[Lam79] Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Trans. Computers*, C-28(9):690–691, 1979.